

Toyota AI Experience Maroc — Documentation Projet

Document de référence pour présentation, soutenance et réponses aux questions du professeur.

Projet : Plateforme web interactive Toyota avec IA, configurateur 3D et back-office admin.

1. Présentation du Projet

Nom

Toyota AI Experience Maroc

Description

Plateforme web moderne dédiée à l'expérience d'achat Toyota au Maroc. Elle combine :

- un **site vitrine premium** (vidéo hero, catalogue, offres),
- un **configurateur 3D** interactif,
- un **chatbot IA** conseiller automobile,
- un **parcours client** (quiz DNA, road trip, achat, concessions),
- un **dashboard administrateur** (leads, réservations, analytics).

Objectif

Moderniser et digitaliser l'expérience d'achat automobile au Maroc : informer, comparer, configurer et convertir les visiteurs en leads qualifiés pour les concessions Toyota.

Public cible

- Clients** : particuliers cherchant un véhicule Toyota
- Administrateurs** : équipes commerciales / managers concession
- Contexte académique** : projet full-stack (frontend, backend, IA, 3D, base de données)

2. Technologies Utilisées

Domaine	Technologies
Frontend	Next.js 16 (App Router), React 19, TypeScript 5
Styles	Tailwind CSS v4, design system Toyota (rouge #EB0A1E, thème sombre)
Animations	Framer Motion v12
3D	Three.js, React Three Fiber, React Three Drei
Base de données	PostgreSQL (Supabase) + Prisma ORM v6
Authentification	NextAuth.js v5 (Credentials + bcrypt)
IA Chatbot	API Groq (modèles Llama) — chat, quiz, road-trip, comparaisons
IA complémentaire	Packages @google/generative-ai / Vercel AI SDK (intégration possible)
ML	Python — microservice FastAPI + scikit-learn (ml-service/)

État global	Zustand (configurateur, comparaison, utilisateur)
Formulaires	React Hook Form + Zod
Cartes	Leaflet / React Leaflet (concessions)
Emails	Nodemailer
Hébergement	Vercel (frontend Next.js), GitHub (code source), Railway (ML optionnel)
Vidéo	Cloudinary (hébergement vidéo hero — évite fichiers lourds dans le repo)
Notifications UI	Sonner (toasts)

Pourquoi ce stack ?

- **Next.js** : SSR/SSG, routing moderne, API Routes intégrées, déploiement simple sur Vercel
- **TypeScript** : typage fort, moins d'erreurs en production
- **Prisma** : ORM type-safe, migrations, seed
- **Groq** : réponses IA rapides et économiques pour le chatbot
- **Three.js** : rendu 3D temps réel dans le navigateur

3. Architecture du Projet

Pattern architectural

Le projet suit l'architecture **Next.js App Router** :

- **Pages** = interface utilisateur (React Server / Client Components)
- **API Routes** = couche serveur REST (`src/app/api/`)
- **Prisma** = couche d'accès aux données (équivalent Modèle + ORM)
- **Composants** = UI réutilisable (`src/components/`)

On peut résumer en **MVC adapté au web moderne** :

- **Modèle** → Prisma + `schema.prisma` + fichiers `src/data/`
- **Vue** → pages et composants React
- **Contrôleur** → API Routes + Server Actions (si utilisées)

Structure des dossiers (simplifiée)

```
toyota-ai-platform/
├─ prisma/
│  └─ schema.prisma      # Schéma base de données
│  └─ seed.ts            # Données initiales (véhicules, admin)
├─ public/
│  └─ images/            # Images véhicules, assets
│  └─ models/            # Modèles 3D GLTF (par véhicule)
├─ ml-service/          # Microservice Python ML (FastAPI)
│  └─ main.py
│  └─ model.py
│  └─ requirements.txt
├─ src/
│  └─ app/
```

			page.tsx	# Page d'accueil
			layout.tsx	# Layout racine
			globals.css	# Styles globaux
			(client)/	# Pages publiques (Header + Footer)
			vehicles/	# Catalogue + fiche véhicule
			configurator/	# Configurateur 3D
			quiz/	# Quiz DNA Toyota
			road-trip/	# Assistant road trip IA
			acheter/	# Parcours d'achat
			concessions/	# Carte concessions
			contact/	# Contact + formulaire
			compte/	# Espace client
			...	
			(admin)/	# Back-office protégé
			dashboard/	
			leads/	
			reservations/	
			analytics/	
			login/	
			api/	# Routes API REST
			auth/	# NextAuth + inscription
			chat/	# Chatbot IA
			leads/	# Gestion leads
			reservations/	# Réservations
			vehicles/	# CRUD véhicules
			quiz/	# Analyse quiz IA
			ml-predict/	# Proxy vers ML Python
			...	
			components/	# Composants UI réutilisables
			layout/	# Header, Footer, AppShell
			catalog/	# Grille véhicules, comparaison
			configurator/	# Scène 3D, panneau config
			chat/	# Widget chatbot
			admin/	# Sidebar, stats, tableaux
			ui/	# Boutons, inputs, design system
			data/	# Données statiques (véhicules, quiz...)
			hooks/	# Hooks React (useChat, etc.)
			lib/	# Utilitaires (auth, groq, geo, email)
			store/	# Zustand stores
			types/	# Types TypeScript
			middleware.ts	# Protection routes admin
			package.json	

API Routes principales

Route	Rôle
POST /api/chat	Chatbot IA (Groq), recommandations véhicules
POST /api/leads	Création d'un lead (formulaire contact / config)
GET/POST /api/reservations	Gestion des réservations essai / RDV

GET /api/vehicles	Liste des véhicules (base Prisma)
POST /api/quiz	Analyse personnalité véhicule (Quiz DNA)
POST /api/road-trip	Génération itinéraire road trip
POST /api/compare	Comparaison IA entre modèles
POST /api/ml-predict	Prédictions marketing (proxy ML Python)
GET/POST /api/auth/[...nextauth]	Authentification NextAuth
POST /api/auth/register	Inscription client
GET /api/analytics	Statistiques dashboard admin

Flux Client / Serveur

```

Navigateur (React)
  ↓ fetch / Server Components
API Routes Next.js (src/app/api/)
  ↓ Prisma Client
PostgreSQL (Supabase)
  ↓
Services externes : Groq API, Cloudinary, ML Service (Python)

```

4. Fonctionnalités Principales

Page d'accueil

- Vidéo hero Toyota (hébergée sur **Cloudinary**)
- Slider véhicules avec animations Framer Motion
- Sections : gamme, offres, CTA vers configurateur et chat IA

Catalogue véhicules (/vehicles)

- Grille de tous les modèles Toyota
- **Filtres** : catégorie (SUV, berline, sport...), recherche, tri prix/puissance
- **Comparaison** : jusqu'à 3 véhicules côte à côte
- Fiches détaillées par modèle (specs, galerie, CTA)

Configurateur 3D (/configurator/[model])

- Rendu **Three.js** du véhicule (fichiers GLTF)
- **Couleur** carrosserie (solid, metallic, pearl)
- **Jantes** et **intérieur** interchangeables
- **Ouverture portes / capot / coffre** (animations charnières)
- Export capture d'écran, simulateur financement, formulaire lead
- Fallback 3D si modèle GLTF absent

Chatbot IA

- Widget flottant accessible sur tout le site
- Conseiller **Toyota AI Advisor** en français
- Recommandations véhicules + liens configurateur / achat

- Historique de conversation côté client (`useChat`)
- Rate limiting pour éviter les abus

Quiz DNA Toyota (`/quiz`)

- Questionnaire personnalité → profil conducteur
- Résultat : véhicule recommandé + carte « Permis de Légende »
- Analyse IA (Groq) ou règles métier en fallback

Road Trip (`/road-trip`)

- Assistant IA pour planifier un road trip au Maroc
- Recommandation véhicule selon l'itinéraire

Parcours Achat (`/acheter`)

- Wizard d'achat, simulateur financement, carte concessions

Concessions (`/concessions`)

- Carte interactive Leaflet
- Géolocalisation, tri par distance

Contact (`/contact`)

- Formulaire lead avec validation
- Coordonnées Toyota Maroc (téléphone, email, adresse, horaires)

Espace client (`/compte`)

- Inscription / connexion
- Profil, configurations sauvegardées, historique leads

Dashboard Admin (`/dashboard` , `/leads` , `/reservations`)

- Statistiques KPI (leads, réservations, conversion)
- Tableau leads avec filtres et recherche
- Gestion réservations essai
- Analytics commerciaux
- Widget prédiction marketing (ML Python)

Authentification

- **Admin** : `admin@toyota-ma.com` (après seed)
- **Client** : inscription libre
- Routes admin protégées par **middleware** + rôle `admin`

5. Base de Données

SGBD

PostgreSQL hébergé sur **Supabase**, accès via **Prisma ORM**.

Tables principales

Table	Description
Vehicle	Véhicules Toyota (nom, catégorie, prix, specs JSON, chemin 3D)

Lead	Demandes clients (contact, config, historique chat, statut)
Reservation	RDV essai / visite lié à un lead
User	Comptes clients (email, mot de passe hashé)
Admin	Comptes administrateurs
Dealership	Concessions (adresse, GPS, horaires)
SavedConfiguration	Configurations 3D sauvegardées par un utilisateur

Relations (simplifié)

```

Vehicle —< Lead >— User
Vehicle —< Reservation
Lead — Reservation (1:1)
Dealership —< Lead
Vehicle —< SavedConfiguration >— User

```

Exemple Prisma (extrait)

```

model Lead {
  id          String    @id @default(cuid())
  firstName   String
  lastName    String
  email       String
  vehicleId   String
  vehicle     Vehicle   @relation(...)
  configuration Json      // Couleur, jantes, options choisies
  chatHistory  Json      // Historique conversation IA
  status       String    @default("new")
  reservation  Reservation?
}

```

Champs JSON

Prisma stocke en `Json` : couleurs, jantes, specs, configuration 3D, historique chat — flexible sans multiplier les tables.

6. Comment lancer le projet

Prérequis

- Node.js 20+
- npm 10+
- Compte Supabase (PostgreSQL)
- Clé API Groq (console.groq.com)
- Python 3.10+ (optionnel, pour le service ML)

Installation

```
# 1. Cloner le projet
git clone <url-du-repo>
cd toyota-ai-platform

# 2. Installer les dépendances
npm install

# 3. Configurer les variables d'environnement
cp .env.example .env.local
# Remplir DATABASE_URL, DIRECT_URL, GROQ_API_KEY, NEXTAUTH_SECRET, etc.

# 4. Générer le client Prisma
npx prisma generate

# 5. Créer les tables et peupler la base
npx prisma db push
npx prisma db seed

# 6. Lancer le serveur de développement
npm run dev
```

Ouvrir <http://localhost:3000>

Variables d'environnement essentielles

Variable	Rôle
DATABASE_URL	Connexion PostgreSQL (pooler Supabase)
DIRECT_URL	Connexion directe (migrations)
GROQ_API_KEY	API IA chatbot / quiz / road-trip
NEXTAUTH_SECRET	Secret session NextAuth
NEXTAUTH_URL	URL de l'app (http://localhost:3000)
ML_SERVICE_URL	URL du microservice Python (optionnel)

Compte admin par défaut (après seed)

- Email : `admin@toyota-ma.com`
- Mot de passe : `Admin@2024!`

Service ML (optionnel)

```
cd ml-service
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
```

Modèles 3D

Placer les dossiers GLTF dans `public/models/<vehicule>/` . Sans modèles, un fallback 3D s'affiche automatiquement.

7. Questions fréquentes du professeur (avec réponses)

Pourquoi Next.js ?

Next.js offre le **rendu côté serveur (SSR)**, le **App Router** moderne, le **code splitting** automatique et des **API Routes** intégrées. Cela améliore les performances, le SEO et simplifie le déploiement sur Vercel.

Pourquoi TypeScript ?

TypeScript ajoute un **typage statique** : erreurs détectées à la compilation, autocomplétion dans l'IDE, code plus maintenable sur un projet full-stack complexe.

Pourquoi Prisma ?

Prisma est un **ORM moderne et type-safe** : le schéma est la source de vérité, le client généré est typé, les migrations et le seed sont simples. Idéal avec PostgreSQL et TypeScript.

Comment fonctionne le chatbot ?

1. L'utilisateur envoie un message via le widget `ChatWidget`
2. Le hook `useChat` appelle `POST /api/chat`
3. L'API construit un **prompt système** avec le catalogue véhicules
4. **Groq** (Llama) génère la réponse en streaming
5. L'API parse les blocs JSON pour **recommandations** ou **comparaisons**
6. L'historique est conservé côté client et peut être joint aux leads

Comment fonctionne le configurateur 3D ?

1. Page `/configurator/[model]` charge `CarScene` (dynamic import, SSR désactivé)
2. **React Three Fiber** crée un `<Canvas>` `Three.js`
3. `CarModel` charge le fichier **GLTF** via `useGLTF`
4. L'utilisateur change couleur/jantes → mise à jour matériaux `Three.js`
5. Animations portes/capot via système de **charnières** (`vehicleHinges`)
6. État global dans **Zustand** (`configuratorStore`)

Quelle est l'architecture globale ?

Architecture **client-serveur** :

- **Frontend** React/Next.js dans le navigateur
- **Backend** API Routes Next.js (REST)
- **Base de données** PostgreSQL via Prisma
- **Services externes** : Groq (IA), Cloudinary (vidéo), ML Python (prédictions)

Comment est gérée la sécurité ?

- **NextAuth.js** : sessions, mots de passe hashés (bcrypt)
- **Middleware** : routes `/dashboard` , `/leads` , `/reservations` réservées aux admins
- **Variables d'environnement** : clés API jamais exposées au client
- **Rate limiting** sur `/api/chat` (anti-abus)
- Rôles utilisateur : `admin` vs `customer`

Pourquoi Tailwind CSS ?

Approche **utility-first** : développement rapide, design cohérent, pas de CSS custom volumineux, responsive facile avec les breakpoints.

Comment sont gérées les images et vidéos ?

- **Images** : composant `next/image` (optimisation, lazy loading, formats modernes)
- **Vidéo hero** : hébergée sur **Cloudinary** (URL distante) pour éviter un repo trop lourd
- Assets statiques dans `public/images/` et `public/models/`

Qu'est-ce que Framer Motion ?

Bibliothèque d'**animations déclaratives** pour React : transitions de pages, hover cards, menus, quiz, configurateur. Utilise `transform` et `opacity` pour de bonnes performances.

Pourquoi Groq et pas uniquement Gemini ?

Le chatbot en production utilise **Groq** (latence faible, gratuit pour le développement). Le projet inclut aussi les SDK Google AI pour d'éventuelles extensions. Le choix Groq répond au besoin de réponses rapides en démo et en production.

Quel est le rôle du service ML Python ?

Microservice **FastAPI** indépendant (`ml-service/`) :

- Entraîne un modèle scikit-learn sur des données marketing
- Prédit type de campagne, canal, ROI
- Appelé par `/api/ml-predict` depuis le dashboard admin

8. Difficultés rencontrées et solutions

Difficulté	Solution adoptée
Fichier vidéo hero trop lourd (> 100 Mo)	Hébergement sur Cloudinary ; <code>preload="metadata"</code> ; pas de vidéo dans le repo Git
Modèles 3D GLTF lourds (20–60 Mo chacun)	Fichiers exclus du repo ; chargement avec Suspense ; fallback 3D animé ; <code>frameLoop="demand"</code> sur le Canvas
Authentification complexe (admin + client)	NextAuth.js v5 + middleware + rôles ; comptes dev en fallback local
Latence chatbot	API Groq (inférence rapide) + rate limiting
Base de données en local	Supabase PostgreSQL gratuit + Prisma seed pour données de test
Performance animations	Framer Motion avec <code>will-change</code> , <code>scaleX</code> au lieu de <code>width</code> , <code>MotionConfig.reducedMotion</code>
Cohérence UI	Design system (<code>BorderDrawButton</code> , tokens Toyota, thème sombre unifié)

9. Commandes utiles

```
npm run dev          # Développement (Prisma generate + Next dev)
npm run build         # Build production
npm run start        # Serveur production
npm run db:setup     # Generate + push + seed
npm run db:studio    # Interface visuelle Prisma
npm run ml:dev       # Lancer le service ML Python
npm run lint         # ESLint
```

10. Équipe & contexte

- **Backend + IA + Architecture** : Badr Eddine Boudhim
- **Frontend + 3D** : Taha

Repository : GitHub — déploiement frontend sur **Vercel**.

Document généré pour le projet Toyota AI Experience Maroc — à utiliser pour révision rapide avant soutenance.